

Summarization: Learning with Contextual Information in Non-Stationary Environments

Summarized by Xiaonan (Nice) Wang *

Summarization[†]Generated based on Paper from Anderson and Hespanha. (2025)

Topic: Boltzmann Learning, Non-Stationary Environments, Learning-based Control, Reinforcement Learning

April 29, 2026

Abstract

This document summarizes the core contributions and methodology of the paper "Learning with Contextual Information in Non-Stationary Environments, Anderson and Hespanha [2025]", focusing on its' main ideas and the core blocks.

1 Overview: Core Questions and Answers

(1) What is the problem?

- **Repeated decision-making** with **contextual (or side) information** in **non-stationary** environments, where the decision-maker lacks a model (or *priori knowledge*) of the *relationship between actions, contexts, and costs*.
 - **(Boltzmann)** Learning based decision-making.
 - Non-stationary Environments: Environmental distributions change with time without a stationary model -> preventing *Bayesian optimal* decision based on past data.
 - 3 elements of modeling:
 - * *Contextual information (Features)*: measurements (data), side information
 - * *Action*: e.g. Control Signal & Dynamic (Robotics)
 - * *Cost & Reward (Optimization Target)*: Costs & Rewards of Actions (Robotics & RL)

(2) Why need to solve this problem?

- In many **real-world** domains, *first-principle* models are unavailable and **learning from data** is necessary.
- Non-stationary: Because involving strategic agents who **observe** and **react** to past decisions, using standard Bayesian optimal decision-making which relies purely on *stationary past* data is unrealistic.

*wangxiaonannice@gmail.com

[†]**Disclaimer:** This summarization is for research and study purposes. It represents a personal interpretation and may contain inaccuracies. Feedback or corrections via email are highly appreciated.

(3) How is it different from prev.?

- Unlike traditional approaches that evaluate average regret over *the whole set of past decisions*, this work introduces a new form of **exponentially weighted "soft window" regret** (i.e. **exponentially decaying weights** of the costs).
 - By the way, soft windows are **more computationally efficient** than *fixed windows*.
- It also broadens the scope of contextual information by **extending** finite measurements to the **continuum** measurement setting (i.e. using the *probability* of measurement's type).

(4) Why is it better than prev.? (Advantages)

- The algorithm provides **explicit bounds on the regret**, showing that **regret can be made arbitrarily small**.
 - Conceptually, the regret is defined as: $R(T) = \Delta(\pi_{actual}, \pi_{static}^*)$
- It is **computationally much more efficient** and requires significantly **less memory** than competing *machine learning approaches* (SVM and Neural Network, which must be *continuously retrained on sliding windows of past data* to handle non-stationarity). -> Due to **"Map"** based Learning (Updating).

(5) What is the approach itself?

- Boltzmann Learning
 - Calculating an **"energy"** for each action based on **past exponentially-discounted costs** associated with the given measurement context;
 - The agent then **stochastically** selects actions using a **Boltzmann distribution** based on these computed energies.
- Novelties
 - Extension of Contextual Information (Measurements) to **Continuum** Spaces
 - **Exponentially Discounted Regret ("Soft Windows")**
 - Highly **Efficient, Lightweight Boltzmann Learning** Architecture
 - Rigorous Theoretical **Regret Bounds (With Probability One)**

(6) What are the applications of it?

- Numerous real-world domains: clinical trials, portfolio selection, pricing, recommender systems, anomaly detection, cyber security, etc.
- Lightweight Learning based Explainable Robotics Control -> Hopefully embedded in real-world robotics systems.

2 The Structure

Summarized Block-Diagram The summarized block-diagram of *Exponentially Discounted Regret based Boltzmann Learning* see fig. 1¹.

¹For efficiency, this diagram was initially hand-drawn on paper and then converted into its current digital version using Nano Banana Pro.

3 Open Questions

1. **Trade-off between Computational Memory Consumption and Learning Performance:** Non-stationary scenarios are actually quite common, for example, CTR prediction in recommendation systems. For such environments, there are many mature solutions even within (training) model-based scopes, such as *online learning*, (*distant time decay*), and *exploration-exploitation strategies*, etc. In the field of applied machine learning, the performance gains brought by (training) model-based approaches seem to fully justify the additional memory consumption required for training. Therefore, does the advantage of “map-based” Boltzmann Learning seem less significant? Or are there other considerations such as a better “alignment” with first principles?

References

- S. Anderson and J. P. Hespanha. Learning with contextual information in non-stationary environments: technical report.
- S. Anderson and J. P. Hespanha. Learning with contextual information in non-stationary environments. 7th Annual Learning for Dynamics & Control Conference (L4DC 2025 ... , 2025.

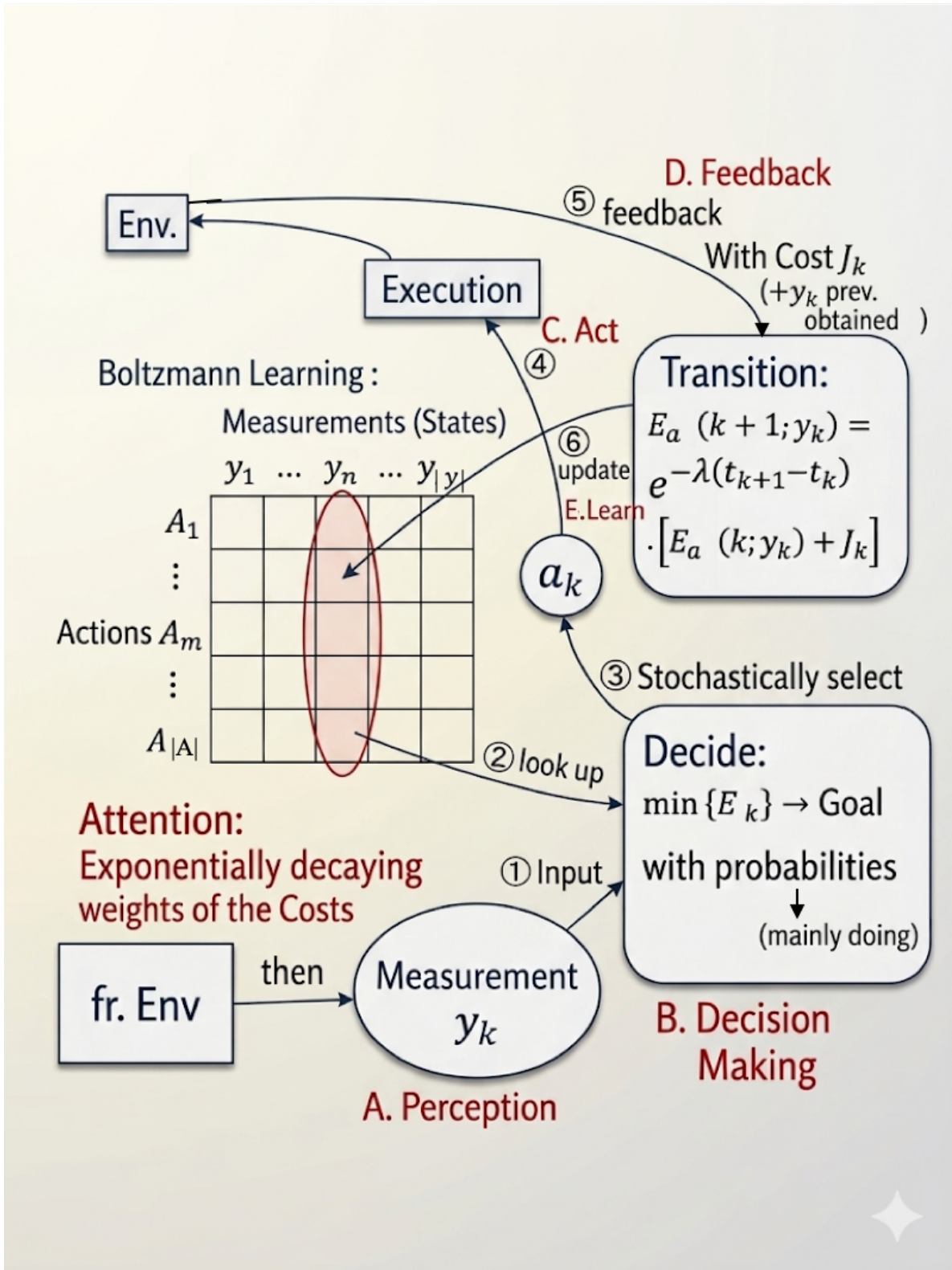


Figure 1: Summarized Block-Diagram of Exponentially Discounted Regret based Boltzmann Learning

A Hand-Drawn Drafts of Mathematical Backbone²

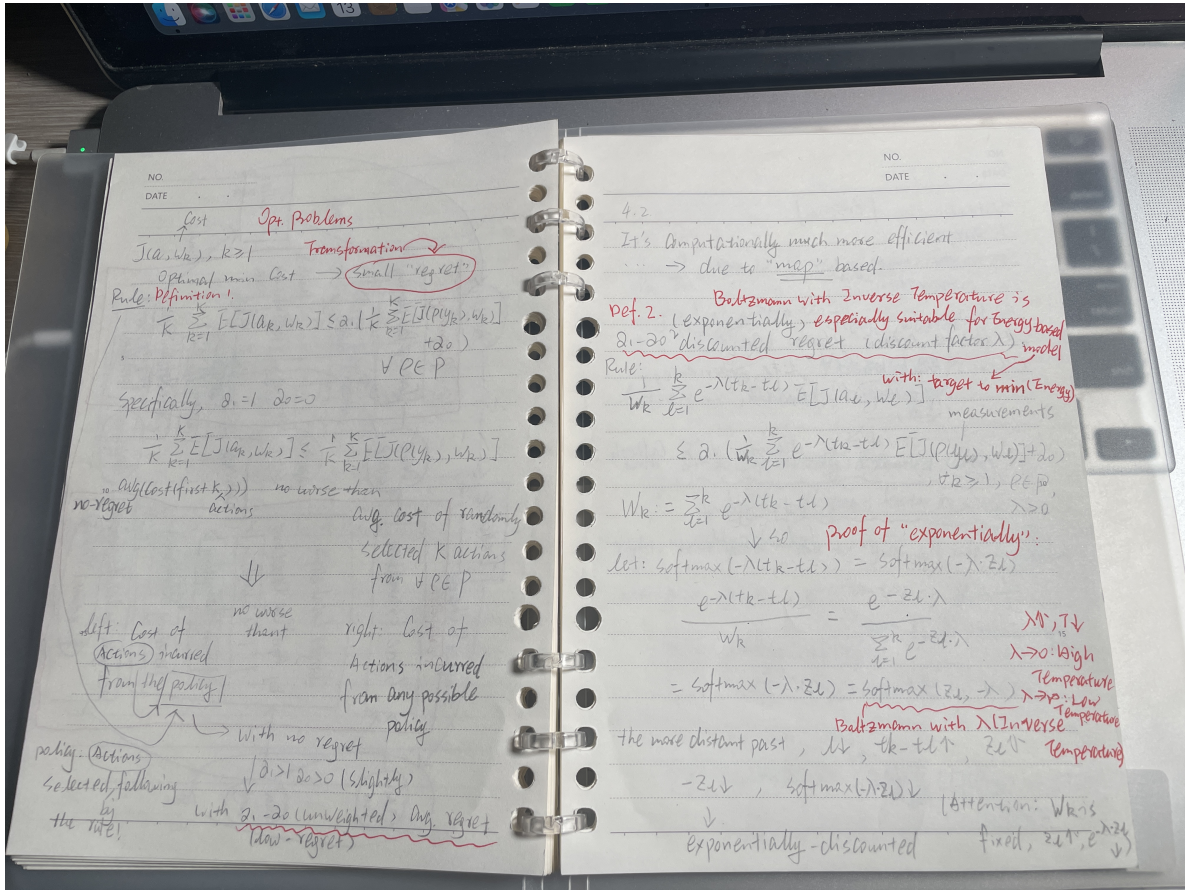


Figure 2: Drafts of Mathematical Backbone - 1: Policy with Low-Regret (Def. 1 + Def. 2), highlighting the Boltzmann Distribution. (PS. Softmax among time-domain)

²This part is mainly used for contributor's notes of understanding.

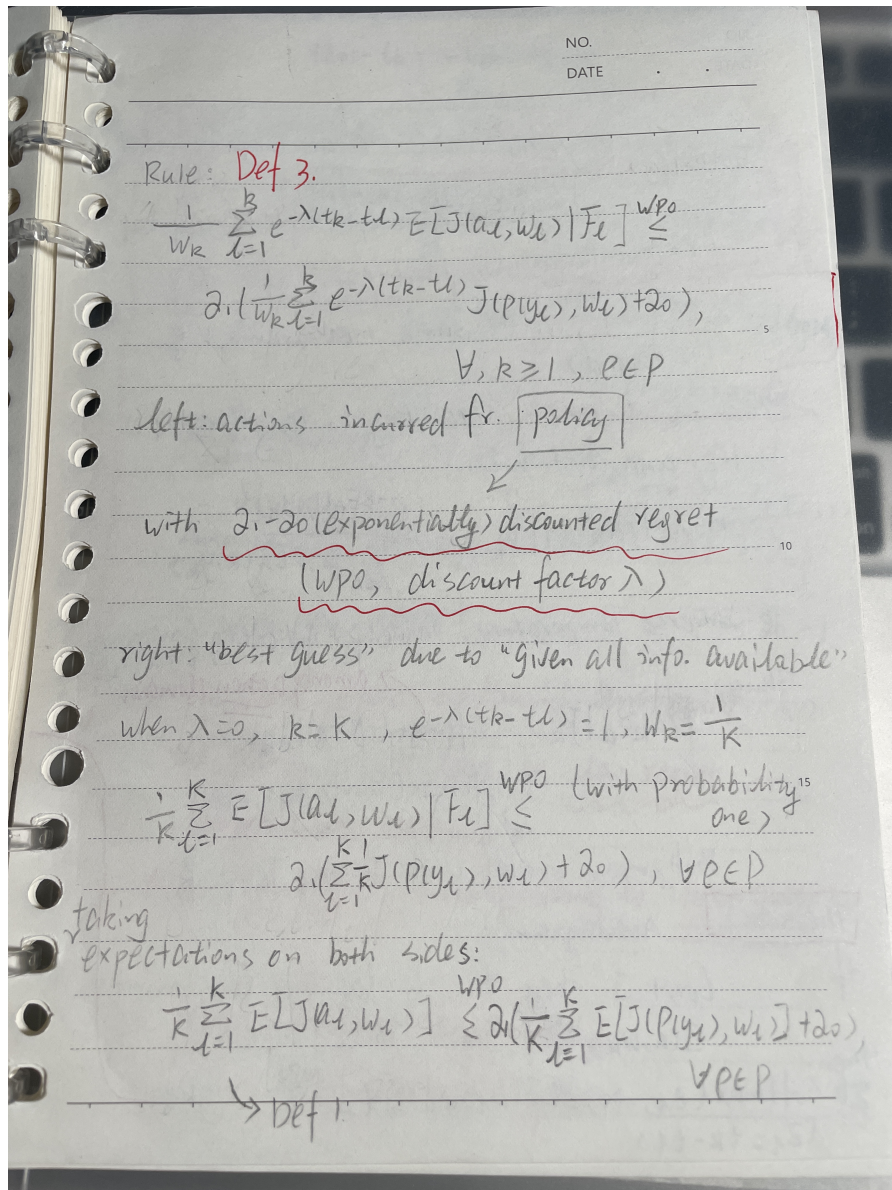


Figure 3: Drafts of Mathematical Backbone - 2

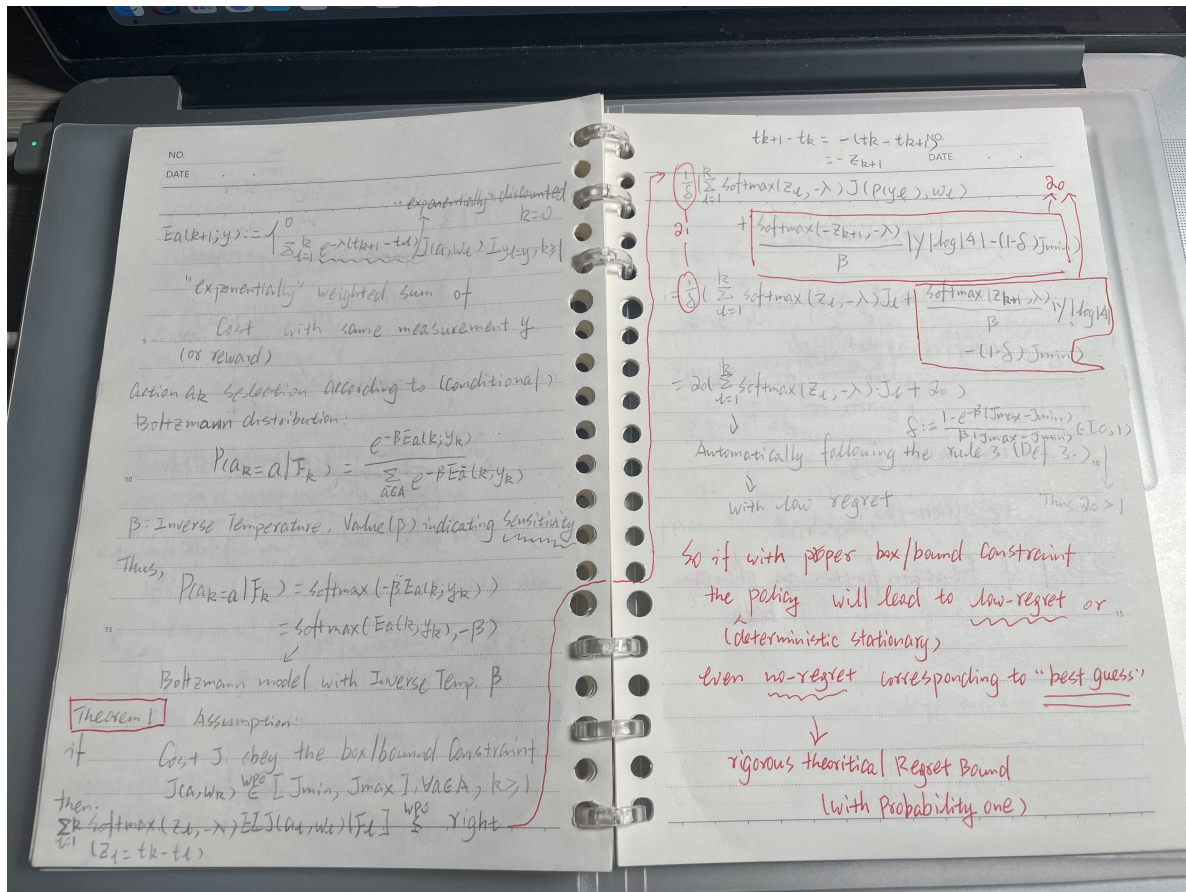


Figure 4: Drafts of Mathematical Backbone - 3: Regarding Theorem 1 - Some Understandings, proof of theorem 1 see Anderson and Hespanha. (PS. Softmax among action-domain)

for $k \geq 1$

$$Ea(k+1; y) = \sum_{i=1}^k e^{-\lambda(t_{k+1}-t_k+t_k-t_i)} J_i I y_i = y$$

$$= e^{-\lambda(t_{k+1}-t_k)} \sum_{i=1}^k e^{-\lambda(t_k-t_i)} J_i I y_i = y$$

$$= e^{-\lambda(t_{k+1}-t_k)} \left\{ \sum_{i=1}^{k-1} e^{-\lambda(t_k-t_i)} J_i I y_i = y \right.$$

$$\left. + e^{-\lambda(t_k-t_k)} J_k I y_k = y \right\}$$

$$= e^{-\lambda(t_{k+1}-t_k)} Ea(k; y)$$

$$+ e^{-\lambda(t_{k+1}-t_k)} J_k I y_k = y$$

→ Proof of Transition function in diagram block

Figure 5: Proof of Transition Function when in Learning.